

Learning Implicit Social Navigation Behavior using Deep Inverse Reinforcement Learning

Tribhi Kathuria, Ke Liu, Junwoo Jang, X. Jessie Yang, and Maani Ghaffari

Abstract—This paper reports on learning a reward map for social navigation in dynamic environments where the robot can reason about its path at any time, given agents’ trajectories and scene geometry. Humans navigating in dense and dynamic indoor environments often work with several implied social rules. A rule-based approach fails to model all possible interactions between humans, robots, and scenes. We propose a novel Smooth Maximum Entropy Deep Inverse Reinforcement Learning (S-MEDIRL) algorithm that can extrapolate beyond expert demos to better encode scene navigability from few-shot demonstrations. The agent learns to predict the cost maps reasoning on trajectory data and scene geometry. The agent samples a trajectory that is then executed using a local crowd navigation controller. We present results in a photo-realistic simulation environment, with a robot and a human navigating narrow crossing scenario. The robot implicitly learns to exhibit social behaviors such as yielding to oncoming traffic and avoiding deadlocks. We compare the proposed approach to the popular model-based crowd navigation algorithm ORCA and a rule-based agent that exhibits yielding. The code and dataset are publicly available at <https://github.com/UMich-CURLY/habicrowd>.

I. INTRODUCTION

The motion and behavior planning of robots are essential components of any real-world robotics stack. Classical planning approaches use simplistic map representations and model humans as static obstacles that are avoided using a reactive planner or controller. While these work well for controlled settings with uncomplicated scene geometry and simplistic interaction scenarios, this approach can fail in more complex task settings.

Artificial Intelligence has recently seen huge advancements with sophisticated vision and language models applied to images, videos, and textual data sourced from internet-derived datasets[1], [2]. The Robotics community has since been looking to use these AI tools on robots, called embodied AI [3]. A prevalent learning paradigm for robots is Reinforcement learning, which aims to instill robotic agents with the capability to autonomously learn and adapt behaviors through interaction with their environment [4], [5].

RL agents rely on handcrafted reward designs for task completion. In the context of social behaviors, these are usually ill-defined, sparse, or biased to design choice given the structure of the problem [6]. While these behaviors are mainly intuitive to humans, it can be hard to encode them as well-defined rewards for RL agents. In social navigation, the most well-studied and benchmarked task is crowd navigation.

The authors are with the University of Michigan, Ann Arbor, MI 48109, USA. {tribhi,kliubiyk,junwoo,xijyang,maanigj}@umich.edu

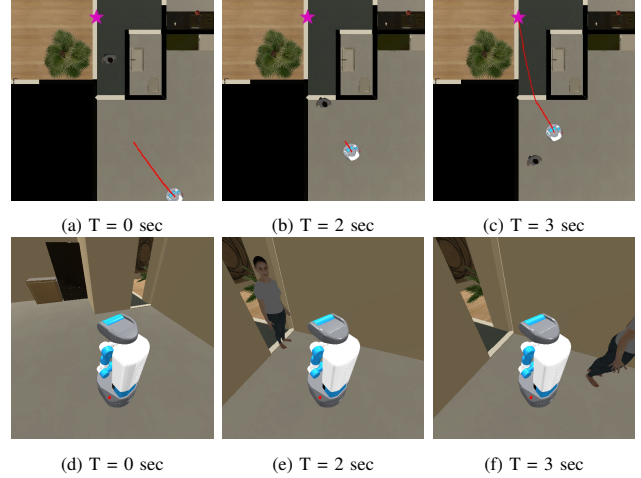


Fig. 1: Robots navigating in dense indoor environments with humans in the scene exhibit implicit social behaviors (legibility, yielding, and so on). We learn from expert demonstration data to teach the robot these implicit navigation behaviors observed over a set of expert demonstrations. Figs. 1a, 1d depict the beginning of an episode, where the expert decides a good position to be on their way to the goal while leaving room for the human. At the intermediary goal, the agent should likely wait for the human to pass and try to avoid it, as seen in Figs. 1b, 1d. Finally, once the human has cleared the path, the agent decides to go to its goal, depicted by the Pink star in Figs. 1c, 1f

[7] propose an RL agent trained on one such handcrafted reward design where the agent is given a large positive reward for successfully navigating to a goal and a small negative reward for getting too close to the dynamic obstacles or collisions. While this is a reasonable reward design and is able to complete the task in a controlled setting well, it is harder to generalize to more complicated task settings. Other improvements on this work, change the input or network design, which then improves performance given the chosen evaluation metrics, adding more bias to the training and evaluation. A perfect social RL agent requires us to theoretically model all interactive behaviors and design reward thresholds to see them in practice. This paper instead proposes an Inverse Reinforcement Learning (IRL) based planner that can learn rewards from a few shot expert demonstrations. IRL has been employed to recover rewards from demonstration successfully for many different tasks in robotics; these include highway lane changing scenarios [8], [9], crowd navigation [10], [11], [12], and manipulation [13]. [14] use Maximum Entropy Deep Inverse Reinforcement Learning (MEDIRL) to learn traversability costmaps for legged robot locomotion. They also propose a trajectory ranking loss to encode a bias towards learning from demonstrations using less robot energy.

We extend the MEDIRL approach to handle dynamic

objects in the presence of complicated scene geometry. We extrapolate demonstration data to allow the agent to learn about image data with few-shot demonstrations [15]. We can extrapolate static trajectories given a map of the environment and add that as extrapolated trajectory data for training the agent. To handle noise added by extrapolation, we propose a smoothing loss gradient that adds embedded navigation information given the robot’s radius, similar to the inflation layer in an occupancy map [16]. We choose an indoor narrow-crossing navigation scenario as a testbed for our agent as depicted in Fig. 1. We train on raw sensor data obtained from the scene, and the agent learns to reason its path, understanding scene geometry and the underlying interaction dynamics implicitly. The agent learns to avoid deadlock and cause minimal disruption to the human trajectory. We compare our approach with a model-based crowd navigation algorithm, Optimal Reciprocal Collision Avoidance (ORCA), as well as a rule-based agent that yields to the human. We use a photo-realistic indoor environment in Habitat-Sim [17] to collect demonstration data using human experts and prepare a pipeline in Robot Operating System (ROS)[18].

In particular, this work makes the following contributions:

- 1) We present a pipeline for training embodied agents to navigate dynamic, cluttered scenes using scene geometry and trajectory data extracted from top-down images.
- 2) We propose an S-MEDIRL agent that has a smoothing loss for learning rewards and uses the extrapolation of demonstration using known scene geometry to learn static and dynamic obstacle avoidance from few-shot demonstrations.
- 3) We present results in a narrow crossing environment and compare our performance with a state-of-the-art crowd navigation controller and a rule-based agent.
- 4) We open-source our dataset and the code that can be used to generate other navigation scenarios using our setup at: <https://github.com/UMich-CURLY/habicrowd>.

II. RELATED WORK

The following section discusses related work in social navigation using both learning and non-learning-based methodologies in Sections II-B, Section II-A. Finally, we introduce work in IRL to lay the foundation for our proposed methodology discussed in Section II-C

A. Non-learning based Social Navigation

Collision avoidance in human-robot interaction has been studied extensively. Some of the earlier methods were model-based. These included the social force model [19] and its extensions, such as extended social force [20], headed social force model [21], and deep social force [22], which use a proxemics-based model to avoid intruding on people’s personal space in robot motion planning. Reciprocal velocity obstacle avoidance is another such approach where the robot builds sets of obstacles based on the observed velocity of other agents and then simultaneously solves for n-paths [23].

These have been adapted to account for different agent dynamics. Another popular reactive planning approach is dynamic window avoidance (DWA), an extension of the A* algorithm [24]. However, a recurring problem with such reactive controllers is that they cannot resolve deadlock in a decentralized manner.

[25] aims to resolve this deadlock in narrow crossing by separating the trajectory plan from the scheduling problem. They use a bi-level optimization algorithm that uses game-theoretic decentralized scheduling of agents given their different priorities (utilities). They, however, assume centralized control. In another work, [26] uses a rule-based agent to exhibit aggressive and assertive ways of resolving right of way through a door-crossing presented user study found that the assertive robot could resolve impasses. [27] model a terminal cost for two differential drive agents that helps avoid deadlock in their socially aware planner. While such model-based methods are explainable, they rely heavily on assumptions about interaction models and do not explicitly address scalability to multiple agents or heterogeneous environments.

B. Deep Learning for Social Navigation

Learning-based methods for social navigation have been primarily studied and benchmarked for the crowd navigation scenario [28]. [7] proposes a deep reinforcement learning-based planner that can avoid collisions with other agents in the environment and minimizes its time to reach the goal simultaneously. The agent is trained in a 10×10 grid world with no obstacles, with other agents navigating using an ORCA model [29]. They define different multi-agent configurations that can cause traditional planners to freeze. Benchmarking this simplistic simulator setup, several studies have shown how to train a deep reinforcement learning agent for crowd navigation.

Socially Aware Reinforcement Learning (SARL) adds a context of social norms to this cost formulation for crowd navigation [30]. They explicitly model crowd dynamics on a coarse grid map and use that to anticipate human behavior for better robot planning. Similarly, [31] distill privileged human trajectory information in their training to have the agent implicitly learn to predict human motion and learn a navigation policy that can successfully avoid a collision. [32] point out the inefficacy of the previously studied methods in deep reinforcement learning for crowd navigation to adapt to different human interaction modules. They define a safety critical reward characterized by a novel Danger One formalization that accounts for all possible human motion given different speeds. They improve the baseline methodologies in RL by adding this safety-critical reward defined on the danger zone.

Other baselines work explicitly for crowd navigation; while most do not consider how to handle static obstacles in the scene, our previous work learns to navigate around humans in the presence of static obstacles [11]. However, we do not consider scene dynamics; rather, we learn to reason about the static obstacle on the robot planning horizon. While this approach works well for a local planning problem, for

successfully avoiding both static and dynamic obstacles in a scene, as we show in our results, these local planners need a reference from a planner that can reason about the scene geometry to handle more complicated environments. This paper presents one such case: the narrow crossing environment with a human walking from the other side, as depicted in Fig. 1. For simplicity and computation time, we do not use a learning-based local planner for our approach, as is discussed in Section VI.

C. Inverse Reinforcement Learning

A popular offshoot of Reinforcement Learning is Inverse Reinforcement learning. The IRL problem is a conjugate of the RL problem where we are given a set of expert demonstrations and need to find the reward function that best explains the demonstration data. For any given set of demonstrations, several rewards can explain human behavior. As a result, research has looked into optimally selecting these reward functions. [33] proposed Maximum Entropy IRL (MaxEnt IRL) did feature matching under the constraint that the distribution over trajectories has maximum entropy. Other approaches for solving the IRL problem include Generative Adversarial Imitation Learning (GAIL) [34], Guided Cost Learning (GCL) [35], Adversarial IRL (AIRL) [36], and Bayesian IRL [37].

The work of [38] generalizes the MaxEnt IRL approach to account for non-linear combinations of the features using neural networks, proposing Maximum Entropy Deep Inverse Reinforcement Learning (MEDIRL).

The work of [39] uses IRL to train autonomous vehicles to merge in highway traffic. They propose a Goal-conditioned SpatioTemporal Zeroing (GSTZ)-MEDIRL framework that can zero out the reward for unvisited states. This is good for learning local control action and is suitable for highway driving, which does not have much dense static information to encode.

IRL has been used in a variety of applications, including manipulation [35], autonomous vehicle control [40], eye-tracking for attention detection [41], route planning [42], trajectory forecasts [43].

In the social navigation context, [44] learn how to approach groups of people using inverse reinforcement learning. The robot learns socially appropriate behavior toward human groups, ensuring natural and acceptable interactions. [12] use a wheelchair robot to learn to navigate a crowded corridor using a global planner to define a path through the corridor. [11], similarly use it for learning to navigate in crowded rooms, using a trajectory rank to learn more from demonstrations that have less jerk in human trajectories. Our work deals with learning social behavior based not only on dynamic agent trajectories but also on scene geometry. As a result, we learn to predict a reward map at any instant, given the images and trajectory data, as opposed to the handcrafted feature design in the previous works.

III. METHODOLOGY

The proposed pipeline learns to navigate cluttered dynamic environments from expert demonstrations, reasoning about

scene geometry and the trajectory of the agents in the scene. We present results on the narrow crossing scenario with one human and a robot trying to navigate from opposite sides and compare them to other model-based methods for crowd navigation. The following section explains our proposed S-MEDIRL algorithm and the data collection for the time-based implementation of the IRL agent to predict different rewards at any time t .

A. Markov Decision Process (MDP)

The IRL agent learns the reward function sampling demonstration data from the expert demonstration buffer. We set the problem as an MDP with the following formulation:

- **State space:** We use a low-resolution top-down image of the environment with grid size 60×60 pixels over $6 \times 6 \text{ m}^2$. The grid is centered on the door position, and the agent positions are known in the grid frame.
- **Action space:** The actions of the agent in the discretized state space are chosen to be $\mathcal{A} = \{\text{up, down, left, right, stay}\}$, which allow the robot to transit from the current grid point to an adjacent grid point.
- **Transition function:** The transition between states is assumed to be deterministic.

Given an MDP, a RL problem seeks the optimal policy $\pi^* : \mathcal{S} \rightarrow \mathcal{A}$, that maximizes the expected discounted reward similar to (1). In the IRL problem, we have a set of Demonstrations ($\mathcal{D}$) on an MDP, and we want to recover the Reward function R . Where demonstrations are a set of observed trajectories τ , which is a sequence of the observed state-action pairs $\tau = \{(s_1, a_1), (s_2, a_2), \dots, (s_T, a_T)\}$. Given a set of features $\phi(s, a)$ that describe the state-action pairs, the reward function $R(s, a)$ can be represented as:

$$R(s, a) = \theta^T \phi(s, a) \quad (1)$$

With this feature-matching algorithm, there are many possible solutions to the problem. Maximum entropy IRL resolves this reward ambiguity by working on the assumption that the demonstrations with higher rewards are exponentially more likely [33].

B. MEDIRL

MEDIRL [38] relaxes the assumption that the reward is a linear function of the features using neural networks to approximate the Reward function.

$$\hat{r}_\theta(s) = f(\phi(s); \theta) \quad (2)$$

The MEDIRL loss is then expressed as follows:

$$\begin{aligned} \mathcal{L}(\theta) &= \log \mathbf{P}(\mathcal{D}, \theta | r_\theta) = \log \mathbf{P}(\mathcal{D} | r_\theta) + \log \mathbf{P}(\theta) \\ &=: \mathcal{L}_\mathcal{D} + \mathcal{L}_\theta, \end{aligned} \quad (3)$$

Where $\mathcal{L}_\mathcal{D}$ is the demonstration loss, $\mathcal{L}_\theta$ is the regularization loss. The loss gradient can then be expressed as follows:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \theta} &= \frac{\partial \mathcal{L}_\mathcal{D}}{\partial \theta} + \frac{\partial \mathcal{L}_\theta}{\partial \theta} = \frac{\partial \mathcal{L}_\mathcal{D}}{\partial r} \cdot \frac{\partial r}{\partial \theta} + \frac{\partial \mathcal{L}_\theta}{\partial \theta} \\ &= (\mu_\mathcal{D} - \mathbb{E}[\mu]) \cdot \frac{\partial r}{\partial \theta} + \frac{\partial \mathcal{L}_\theta}{\partial \theta}. \end{aligned} \quad (4)$$

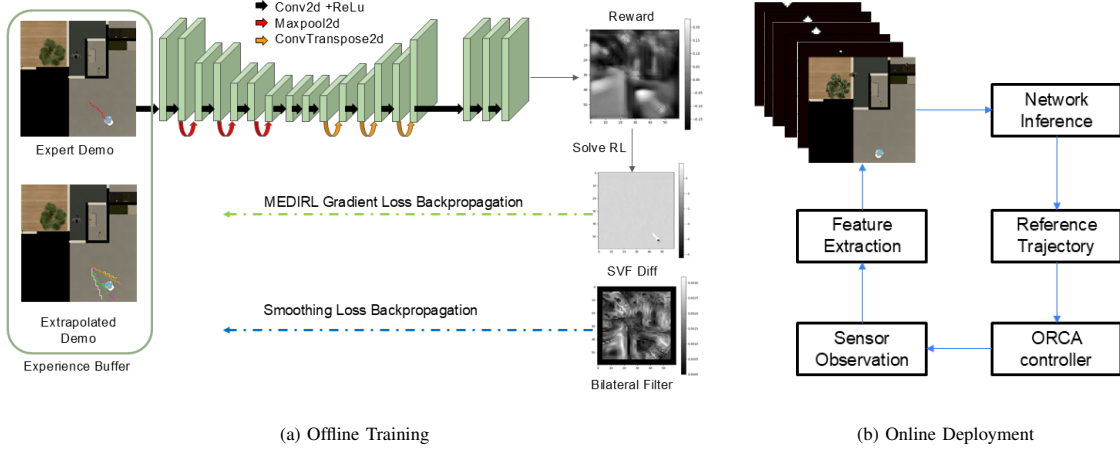


Fig. 2: Fig. 2a shows the training architecture. The expert demos are sampled every 0.1s to create a new demonstration data point. The past trajectories of the human and robot are fed in to give the network history of past positions. The U-Net architecture feeds into the final regression layer, which outputs the learned reward. The reward is then used to sample a trajectory (E_μ), which is compared to the demonstrated trajectory at any time t , looking 10 steps in the future. This gradient is then backpropagated till the network converges. Fig. 2b shows the online deployment pipeline, where the reward inferred using the network is used to generate the reference trajectory for the local controller.

The first part of this sum can be computed as the difference between the expert and the sampled trajectory from the learned reward function, as can be seen in Fig. 2a. The second is backpropagated as a regularization loss of the weights of the network.

C. Smooth MEDIRL

MEDIRL has been widely used to infer reward functions in static environments. This works well, as the agent’s trajectory, given a scene, remains almost the same. As a result, the IRL agent can reason about scene geometry to infer characteristics such as traversability [14]. On the other hand, for the dynamic navigation task, the reward depends on the trajectory data of the agents. When navigating cluttered dynamic environments, both these problems get convolved together.

To explicitly add navigability information to our network without increasing data collection overhead, we propose to extrapolate our demonstration data. The expert data consists of 15 demonstration data points collected over seven different start and goal positions of the robot and human in the scene. This is, however, insufficient to capture scene navigation as it leaves many states unseen; as a result, we extrapolate to our demonstration data depicted in Fig. 3. We extrapolate the current trajectory data by giving it six random start positions close to the current start of the robot. This helps the network generalize better to unseen test cases.

To incorporate temporal information into the grid environment, explicit stopping points, or counter-stops, are added to the demonstration data. On a discrete grid with a fixed look-ahead, trajectories may appear to move even when the robot is stationary. To address this, we set a 3-second threshold to identify counter-stops, excluding states where the controller is merely turning in place. The reference trajectory spans 10 grid points, and data near counter-stops is augmented with stopping locations to help the robot learn to pause and wait as demonstrated. Fig. 3 shows the robot stopping

at the first counter-stop. Fig. 4b shows an instance of the

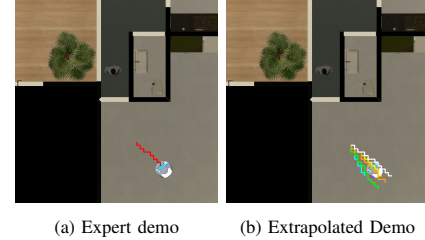


Fig. 3: The figure shows the augmented noise data fed into the network for one of our trajectories. Fig. 3a depicts the expert demo, while Fig. 3b depicts the added noisy trajectory data.

robot navigating an unseen test case using the reward trained with noisy data. We note many added aberrations due to the added noise, where neighboring pixels have different reward values. The noise information being randomly sampled can bias the system to create some local minima and maxima in the reward. We propose a smoothing loss that can inform the network of the scene traversability while eliminating the aberrations. The proposed loss, implemented as Bilateral Filtering loss, is applied to the trained reward values to ensure that the reward function is smooth and that the reward can generalize across similar states. Compared to the general robotics stack, this loss emulated the inflation layer of a cost map in our learning-based framework.

The Bilateral filtering loss is defined in (5) and is added to the MEDIRL loss given in (3).

$$\mathcal{L}_{\text{bilateral}}(r) = \sum_p \sum_{q \in \mathcal{N}(p)} \|r(p) - r(q)\|^2 \exp\left(-\frac{\|p - q\|^2}{2\sigma_s^2}\right) \cdot \exp\left(-\frac{\|r(p) - r(q)\|^2}{2\sigma_c^2}\right). \quad (5)$$

Here, $\mathcal{N}(p)$ represents the neighborhood of a pixel p , and q is a neighboring point in $\mathcal{N}(p)$. The terms σ_s and σ_c control the sensitivity to spatial distance ($\|p - q\|$) and residual

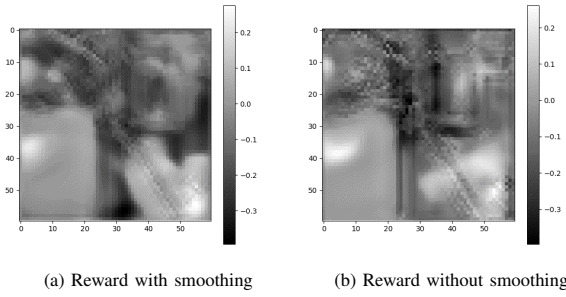


Fig. 4: Comparison of a reward at time t trained with and without the smoothing loss. Fig. 4a shows that there are fewer pixels allocated starkly different values to its neighbors compared to Fig. 4b

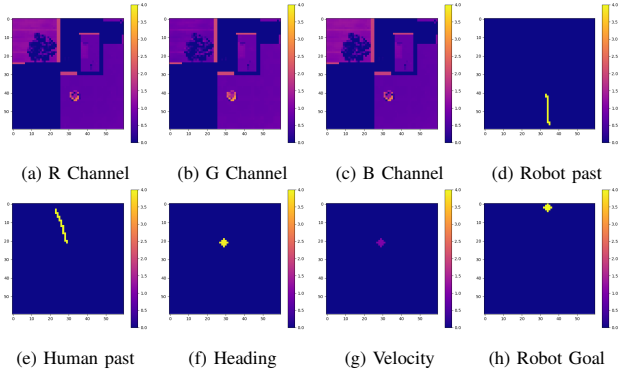


Fig. 5: This figure provides a snapshot of the network's input features. The first three features, extracted from the RGB image of the demonstration's top-down view, are normalized and split into Red, Green, and Blue channels (Fig. 5a-5c). The network also receives the episode history, including the robot's past trajectory (Fig. 5d) and the human's past trajectory (Fig. 5e). The human's current state, represented by velocity and heading (Fig. 5g-5f), and the robot's goal (Fig. 5h) are also encoded as inputs.

difference ($\|r(p) - r(q)\|$), respectively, ensuring smoothness that respects both spatial and intensity variations.

Fig. 4 shows the reward learned using the smoothing loss compared to the one learned without smoothing. There are fewer pixelated areas where the robot can be stuck in a local maxima.

D. Training and Online Deployment

The network input is a stacked feature set consisting of the images seen in Fig. 5. The features are collected every 0.1s apart, and the reference trajectory for each feature is extracted for 10 discrete steps in the future. The RGB image feature is divided into three normalized image features, each representing the RGB values respectively seen in Fig. 5a-5c. Other features are extracted from agents position; these include the human's past trajectory, seen in Fig. 5e, and robots past trajectory in Fig. 5d. We also consider human dynamics such as heading and velocity depicted in Fig 5f and Fig. 5g, respectively. Finally, we condition the training to the robots goal as seen Fig. 5h, which is assumed to be known, which is the case for most planning problems.

As seen in Fig 2b, the network is then deployed to predict the reward given the top-down image and the human trajectory in the observed scene. The reward is used to then get a trajectory for the robot, resampled every 0.2 seconds. We use an ORCA controller to follow this reference. We

present results on 13 unseen cases in the same scene as test results. We have a U-Net architecture depicted in Fig. 2a similar to [14]. The robot learns to adapt to different start and goal positions of both the human and the robot and implicitly predicts how to yield to the human and when to start moving to the goal.

IV. EXPERIMENTS

We present results collected in a Habitat-Sim simulation environment. We fix a scene setup and assign different start and goal positions to both the human and robot for each episode. For testing our learned agent, we have 13 unseen episodes and deploy the policy replanning the trajectory every 0.2s. The learned trajectory is then given to an ORCA controller that can avoid collisions between dynamic agents and return a velocity that the robot can execute. We use ROS to communicate between modules, convert images to feature space, infer sampled robot trajectory from predicted reward cost map, and finally, execute the reference trajectory using ORCA.

A. Baselines

We evaluate two baselines relevant to our task:

1) *ORCA Agent*: The first baseline is an Optimal Reciprocal Collision Avoidance (ORCA) agent, which uses a reference trajectory generated by a shortest path planner on a static map. This planner operates on navigation mesh edges, similar to a Voronoi planner, as detailed in [45]. While ORCA inherently avoids dynamic collisions, it relies on the higher-level planner to account for static obstacles. In our implementation, the ORCA agent replans every 0.2 seconds with a discrete step size of 1.33 m/s and a 4-second planning horizon. The maximum speed is 1.3 m/s, and dynamic agents are considered within a 0.5 m horizon. Static obstacles can occasionally constrain ORCA, causing deviations from its intended path.

2) *ORCA-Backoff Agent*: The second baseline introduces a rule-based backoff mechanism to address deadlocks. It builds upon the ORCA agent in Section IV-A.1 by adding a deadlock detection mechanism: if two agents move less than 0.1 m over 50 timesteps (10 seconds). In such cases, the agent retreats to its start position, waits for the human agent to pass, and then proceeds.

B. Results

To test the performance of our IRL agent, we compare it against the baselines in a narrow crossing setup. The human agent is walking from the other side of the door and trying to pass the robot on the other side. The human agent does not back off and has priority in the path; the robot agent must either try to go to the goal or back off and yield to the human until the path is clear and then move towards its goal.

Fig. 6 shows the rate of success of our agent (S-MEDIRL) compared to the baselines collected over 5 runs for each 13 episodes. The ORCA agent, getting its reference path from the shortest path planner, ends up in a deadlock for all the

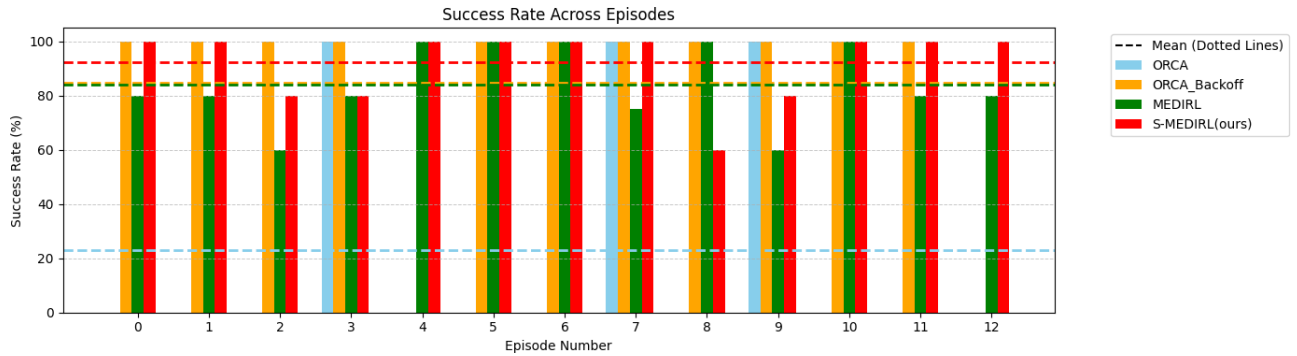
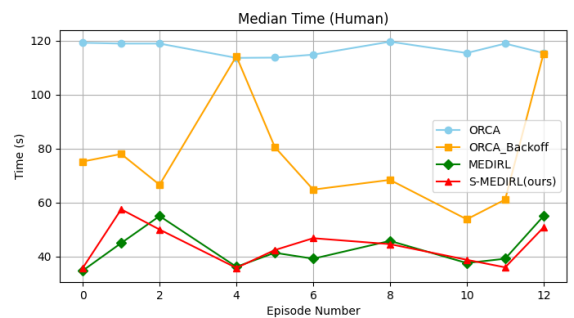


Fig. 6: The figure shows the success rate of the MEDIRL agents compared to our model-based (ORCA) and rule-based (ORCA) baselines collected over 5 runs. For episodes that do not have deadlock and no negotiation involved in passing the narrow crossing (3, 7, and 9), all agents perform well and are able to complete the task. For episodes where the agents end up in deadlock, the nominal ORCA agent fails, but the backoff agent is able to succeed in most cases. The failure in the backoff case comes from the starting position not always being the best position to yield. The MEDIRL agent is prone to end up in local minima if it gets stuck in unseen states in the scene, and as result has a median success rate of about 82%, similar to the ORCA backoff agent. While, our S-MEDIRL is able to successfully navigate the scene 92% of the time.

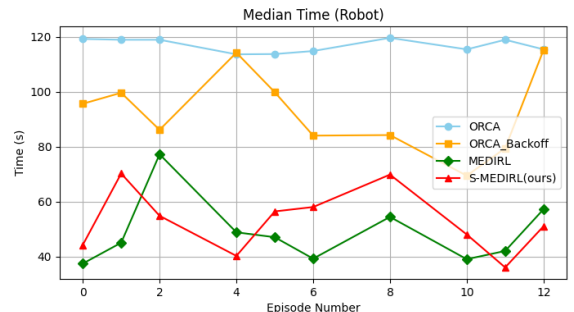
scenarios except episodes 3, 7, and 9. In case of no deadlock, this agent has a perfect success rate, as is expected. However, when there is a deadlock, the agents fail to complete the task. The ORCA backoff agent is able to complete most tasks except for 4 and 12. This agent is trained to choose the robot's start position to yield to the human, which in some cases, such as in episodes 4 and 12, are in the human's way, and it fails to let the human pass. The MEDIRL agent can successfully complete each scene but has some failure cases where it gets stuck in local minima or unseen states. The average success rate for this agent is similar to the Backoff agent, about 82%. The S-MEDIRL agent can recover from most unseen states and bring up the average success rate to about 92%.

Qualitatively the agent's performance for a selected run of episode 8 can be seen in Fig. 8. The S-MEDIRL agent is at the goal in under a minute, depicted in a discrete time of $t = 44$ seconds. The MEDIRL agents get stuck in a local minima for around $t = 30$ and end up taking longer to complete the task. The ORCA.backoff agent finally completes the task at $t = 89$ seconds. As can be seen through the time-lapse, this agent first tries to go to the goal and end up in deadlock at $t = 15$ seconds. Then, it decides to back off and goes to its start position, seen in $t = 30$ to $t = 59$ seconds. Once the human has cleared at $t = 59$ seconds, the agent finally decides to move to the goal. The IRL agents instead avoid this deadlock by waiting for the human to pass first and leaving room for the human, as seen in $t = 15$ s to $t = 30$ s, and then start to move to its goal when the path is clear. The ORCA agent ends up in a deadlock and cannot finish the task.

Quantitatively, we evaluate the quality of the trajectory of our agent by looking at the median time taken by the human and robot to complete the task over the 5 runs on each episode. Fig. 7 describes the time taken for both the human and the robot to complete their task, i.e., reach their respective goals for navigating all episodes with deadlock. As is seen for the success metrics in Fig. 6, the ORCA agent fails to complete the task; this is reported in Fig. 7 as the ORCA agent times out at 2 minutes of task completion time.



(a) Time taken by the human to complete the episodes with deadlock



(b) Time taken by the robot to complete the episodes with deadlock.

Fig. 7: The figure depicts the overall time taken by the robot and the time taken to reach their respective goals. For clarity, we only include the cases where deadlock happens, so the yielding action by the robot is required. Fig. 7a depicts the time taken by the human to complete the task when the robot is operating using different algorithms. We note that while ORCA baseline fails for all these and times out at about 120 seconds. The ORCA backoff agent succeeds for all but two cases, reported in Fig. 6, but in general takes about 20 seconds extra for the human and 30 seconds extra for the robot. (Fig. 7b)

The ORCA backoff agent can complete the task; however, it takes about 30 seconds longer on average than the IRL agents, as the yielding position is arbitrary.

V. CONCLUSION

This paper presents a MEDIRL-based agent for navigating indoor environments with complex scene geometry with humans. The proposed agent is trained on top-down images of the scene, the agent's trajectories, human velocity, heading,

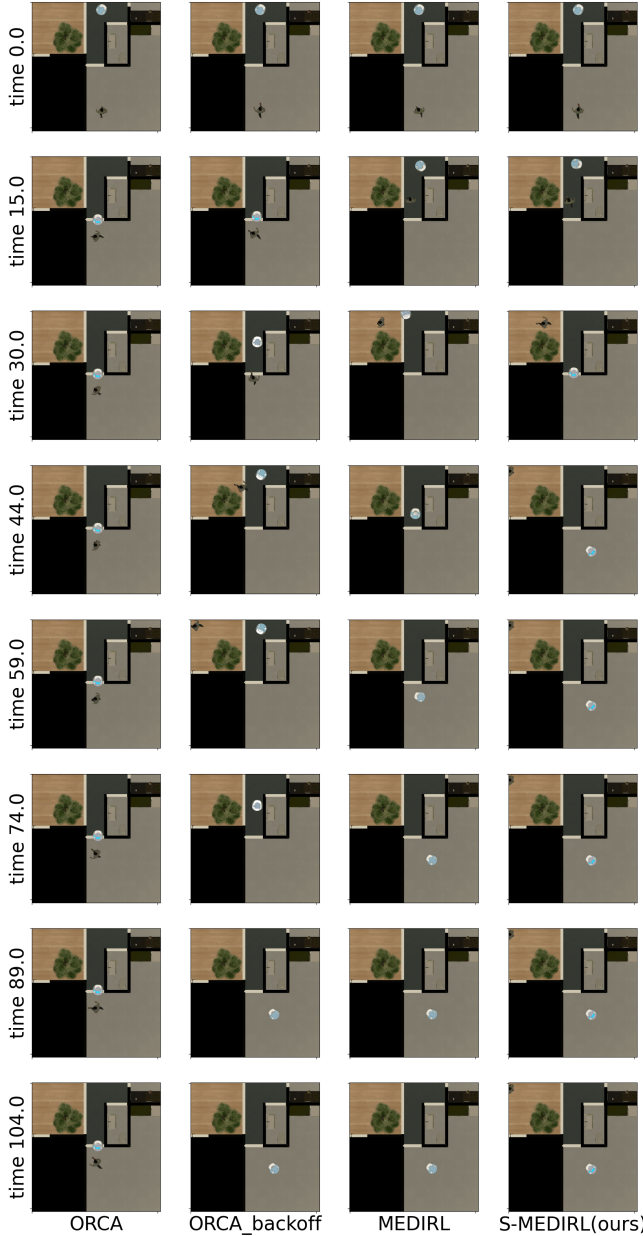


Fig. 8: This figure shows snippets of the robot trajectory at discrete time instants. The agents position is noted, relative to the humans position and goal. The ORCA agent fails in the given scenario, and the agents end up in a deadlock. The ORCA-Backoff agent can successfully resolve the deadlock, but takes longer to complete the task, than the IRL agents. The MEDIRL is more prone to being stuck in local minima and ends up taking longer.

and robot goal. Training on this image data takes away the reliance on handcrafted features that can be arbitrary to different robots’ tasks and design preferences. The agent learns from human demonstration how to yield to oncoming traffic and go forward through a narrow crossing. We compare the performance of the agent with that of a nominal ORCA agent and an ORCA-backoff agent. We saw that the IRL agents were implicitly able to learn to avoid deadlock and could complete the task faster than the baseline comparisons. To improve the agent performance when it gets stuck in unseen parts of the scene, we propose a way to extrapolate demon-

stration data and a smoothing loss to eliminate aberrations in training the reward. The S-MEDIRL shows comparable task completion times with improved success rate of all tasks as compared to the MEDIRL agent. We emphasize that while it is hard to account for desired social behavior, we may want the agent to exhibit that learning from demonstration can help us learn these norms and goals end-to-end.

VI. DISCUSSION

The purpose of this study is to shed light on the need to learn social behavior from demonstrations. We describe how social behavior is implicitly a part of our trajectory planning as humans and is hard to model in an exhaustive list for social agents to be able to accomplish in a rule-based manner. We show that learning this behavior from the demonstration of raw image data removes any design biases that may come from handcrafted feature design for the completion of specific tasks. We never explicitly work on detecting or avoiding deadlock and observe that the agent can implicitly learn deadlock avoidance behavior from demonstration data. However, like most agents that rely on demonstrations for learning motion, we rely heavily on there being good demonstrations available. We also find that this qualitative good metric is hard to quantify and differs for different tasks. Previous approaches have fixed what these qualitative goodness metrics are and account for preference learning. Our approach learns these biases, assuming good demonstration data is available.

Our future work wants to extend our approach to encode nominal goodness metrics into our system design. We want to use a Vision and Language model to predict the goodness of a demonstration and then encode these preferences into our agent as a trajectory rank similar to [14].

ACKNOWLEDGMENT

We would like to thank our collaborators at the University of Michigan Museum of Art, Grace VanderVliet and John Turner, for their constant support through our progress.

REFERENCES

- [1] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, “Learning transferable visual models from natural language supervision,” in *International Conference on Machine Learning*, 2021. [Online]. Available: <https://api.semanticscholar.org/CorpusID:231591445>
- [2] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” *Proc. IEEE Conf. Comput. Vis. Pattern Recog.*, pp. 10 674–10 685, 2021. [Online]. Available: <https://api.semanticscholar.org/CorpusID:245335280>
- [3] J. Duan, S. Yu, H. L. Tan, H. Zhu, and C. Tan, “A survey of embodied ai: From simulators to research tasks,” *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 6, no. 2, pp. 230–244, 2022.
- [4] M. Savva, A. Kadian, O. Maksymets, Y. Zhao, E. Wijmans, B. Jain, J. Straub, J. Liu, V. Koltun, J. Malik, et al., “Habitat: A platform for embodied ai research,” in *Proc. IEEE Int. Conf. Comput. Vis.*, 2019, pp. 9339–9347.
- [5] J. Kober, J. A. Bagnell, and J. Peters, “Reinforcement learning in robotics: A survey,” *Int. J. Robot. Res.*, vol. 32, no. 11, pp. 1238–1274, 2013.

- [6] U. Jain, I.-J. Liu, S. Lazebnik, A. Kembhavi, L. Weihs, and A. G. Schwing, "Gridtopix: Training embodied agents with minimal supervision," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2021, pp. 15 141–15 151.
- [7] Y. F. Chen, M. Liu, M. Everett, and J. P. How, "Decentralized non-communicating multiagent collision avoidance with deep reinforcement learning," in *Proc. IEEE Int. Conf. Robot. and Automation*. IEEE, 2017, pp. 285–292.
- [8] J. Liu, L. N. Boyle, and A. G. Banerjee, "An inverse reinforcement learning approach for customizing automated lane change systems," *IEEE Transactions on Vehicular Technology*, vol. 71, no. 9, pp. 9261–9271, 2022.
- [9] S. B. Prathiba, G. Raja, and N. Kumar, "Intelligent cooperative collision avoidance at overtaking and lane changing maneuver in 6g-v2x communications," *IEEE Transactions on Vehicular Technology*, vol. 71, no. 1, pp. 112–122, 2021.
- [10] R. Wang, W. Wang, and B.-C. Min, "Feedback-efficient active preference learning for socially aware robot navigation," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2022, pp. 11 336–11 343.
- [11] Y. Xu, T. Chakhachiro, T. Kathuria, and M. Ghaffari, "Solo t-dirl: Socially-aware dynamic local planner based on trajectory-ranked deep inverse reinforcement learning," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 12 045–12 051.
- [12] B. Kim and J. Pineau, "Socially adaptive path planning in human environments using inverse reinforcement learning," *International Journal of Social Robotics*, vol. 8, pp. 51–66, 2016.
- [13] E. Biyik, N. Huynh, M. J. Kochenderfer, and D. Sadigh, "Active preference-based gaussian process regression for reward learning," *arXiv preprint arXiv:2005.02575*, 2020.
- [14] L. Gan, J. W. Grizzle, R. M. Eustice, and M. Ghaffari, "Energy-based legged robots terrain traversability modeling via deep inverse reinforcement learning," *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 8807–8814, 2022.
- [15] D. S. Brown, W. Goo, P. Nagarajan, and S. Niekum, "Extrapolating beyond suboptimal demonstrations via inverse reinforcement learning from observations," 2019. [Online]. Available: <https://arxiv.org/abs/1904.06387>
- [16] D. V. Lu, D. Hershberger, and W. D. Smart, "Layered costmaps for context-sensitive navigation," in *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2014, pp. 709–715.
- [17] X. Puig, E. Undersander, A. Szot, M. D. Cote, T.-Y. Yang, R. Partsey, R. Desai, A. W. Clegg, M. Hlavac, S. Y. Min, *et al.*, "Habitat 3.0: A co-habitat for humans, avatars and robots," *arXiv preprint arXiv:2310.13724*, 2023.
- [18] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, "Ros: an open-source robot operating system," in *ICRA workshop on open source software*, vol. 3, no. 3.2. Kobe, Japan, 2009, p. 5.
- [19] D. Helbing and P. Molnar, "Social force model for pedestrian dynamics," *Physical Review E*, vol. 51, 05 1998.
- [20] M. Moussaïd, N. Perozo, S. Garnier, D. Helbing, and G. Theraulaz, "The walking behaviour of pedestrian social groups and its impact on crowd dynamics," *PloS one*, vol. 5, no. 4, p. e10047, 2010.
- [21] F. Farina, D. Fontanelli, A. Garulli, A. Giannitrapani, and D. Praticchizzo, "Walking ahead: The headed social force model," *PloS one*, vol. 12, no. 1, p. e0169734, 2017.
- [22] S. Kreiss, "Deep social force," 2021.
- [23] J. Van den Berg, M. Lin, and D. Manocha, "Reciprocal velocity obstacles for real-time multi-agent navigation," in *Proc. IEEE Int. Conf. Robot. and Automation*. Ieee, 2008, pp. 1928–1935.
- [24] D. Fox, W. Burgard, and S. Thrun, "The dynamic window approach to collision avoidance," *IEEE Robot. Autom. Mag.*, vol. 4, pp. 23–33, 1997.
- [25] R. Chandra, R. Menon, Z. Sprague, A. Anantula, and J. Biswas, "Decentralized social navigation with non-cooperative robots via bi-level optimization," *arXiv preprint arXiv:2306.08815*, 2023.
- [26] J. Thomas and R. Vaughan, "After you: doorway negotiation for human-robot and robot-robot interaction," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 3387–3394.
- [27] S. H. Arul, J. J. Park, and D. Manocha, "Ds-mpepc: Safe and deadlock-avoiding robot navigation in cluttered dynamic scenes," *arXiv preprint arXiv:2303.10133*, 2023.
- [28] C. Mavrogiannis, F. Baldini, A. Wang, D. Zhao, P. Trautman, A. Steinfield, and J. Oh, "Core challenges of social robot navigation: A survey," *ACM Transactions on Human-Robot Interaction*, vol. 12, no. 3, pp. 1–39, 2023.
- [29] J. Van Den Berg, S. J. Guy, M. Lin, and D. Manocha, "Reciprocal n-body collision avoidance," in *Robotics Research: The 14th International Symposium ISRR*. Springer, 2011, pp. 3–19.
- [30] C. Chen, Y. Liu, S. Kreiss, and A. Alahi, "Crowd-robot interaction: Crowd-aware robot navigation with attention-based deep reinforcement learning," in *2019 international conference on robotics and automation (ICRA)*. IEEE, 2019, pp. 6015–6022.
- [31] G. Monaci, M. Aractingi, and T. Silander, "Dipcan: Distilling privileged information for crowd-aware navigation," in *Robotics: Science and Systems*, 2022.
- [32] S. S. Samsani and M. S. Muhammad, "Socially compliant robot navigation in crowded environment by human behavior resemblance using deep reinforcement learning," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 5223–5230, 2021.
- [33] B. D. Ziebart, A. L. Maas, J. A. Bagnell, A. K. Dey, *et al.*, "Maximum entropy inverse reinforcement learning," in *Proc. AAAI Nat. Conf. Artif. Intell.*, vol. 8. Chicago, IL, USA, 2008, pp. 1433–1438.
- [34] J. Ho and S. Ermon, "Generative adversarial imitation learning," 2016. [Online]. Available: <https://arxiv.org/abs/1606.03476>
- [35] C. Finn, S. Levine, and P. Abbeel, "Guided cost learning: Deep inverse optimal control via policy optimization," in *International conference on machine learning*. PMLR, 2016, pp. 49–58.
- [36] J. Fu, K. Luo, and S. Levine, "Learning robust rewards with adversarial inverse reinforcement learning," 2018. [Online]. Available: <https://arxiv.org/abs/1710.11248>
- [37] D. Ramachandran and E. Amir, "Bayesian inverse reinforcement learning," in *IJCAI*, vol. 7, 2007, pp. 2586–2591.
- [38] M. Wulfmeier, P. Ondruska, and I. Posner, "Maximum entropy deep inverse reinforcement learning," *arXiv preprint arXiv:1507.04888*, 2015.
- [39] K. Lee, D. Isele, E. A. Theodorou, and S. Bae, "Spatiotemporal costmap inference for mpc via deep inverse reinforcement learning," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 3194–3201, 2022.
- [40] C. You, J. Lu, D. Filev, and P. Tsotras, "Advanced planning for autonomous vehicles using reinforcement learning and deep inverse reinforcement learning," *Robotics and Autonomous Systems*, vol. 114, pp. 1–18, 2019.
- [41] S. Bae, E. Pakdamanian, I. Kim, L. Feng, V. Ordonez, and L. Barnes, "Medirl: Predicting the visual attention of drivers via maximum entropy deep inverse reinforcement learning," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 13 178–13 188.
- [42] Z. Zhao and Y. Liang, "A deep inverse reinforcement learning approach to route choice modeling with context-dependent rewards," *Transportation Research Part C: Emerging Technologies*, vol. 149, p. 104079, 2023.
- [43] N. Deo and M. M. Trivedi, "Trajectory forecasts in unknown environments conditioned on grid-based plans," *arXiv preprint arXiv:2001.00735*, 2020.
- [44] Y. Gao, F. Yang, M. Frisk, D. Hernandez, C. Peters, and G. Castellano, "Learning socially appropriate robot approaching behavior toward groups using deep reinforcement learning," in *2019 28th IEEE international conference on robot and human interactive communication (RO-MAN)*. IEEE, 2019, pp. 1–8.
- [45] T. Kathuria, Y. Xu, T. Chakhachiro, X. J. Yang, and M. Ghaffari, "Providers-clients-robots: Framework for spatial-semantic planning for shared understanding in human-robot interaction," in *2022 31st IEEE International Conference on Robot and Human Interactive Communication (RO-MAN)*. IEEE, 2022, pp. 1099–1106.